

فاز ۷: سیستم عامل (Operating System) –

ویژه بکاند کاران

اینجا هدف این نیست که به ادمین لینوکس بشی، بلکه هدف اینه که بفهمی برنامه‌ای که می‌نویسی (مثلاً به اپلیکیشن Spring Boot) چطور با سیستم عامل تعامل می‌کنه، حافظه رو مدیریت می‌کنه، و درخواست‌های همزمان رو پردازش می‌کنه. مصاحبه‌کننده می‌خواد ببینه درک عمیقی از محیط اجرای کدت داری.

7.1 پردازش (Process) و ریس (Thread)

این پایه‌ای‌ترین و مهم‌ترین مفهوم در سیستم عامله که مستقیماً به جاوا ربط داره.

! سوال مصاحبه‌ای ۱: تفاوت پردازش (Process) و ریس (Thread) رو کامل توضیح بده

جواب:

• پردازش (Process): به برنامه در حال اجراست. وقتی Spring Boot رو اجرا می‌کنی، به Process ساخته میشه. هر Process دارای فضای حافظه مستقل و مجزا هست (شامل Heap، Stack، و داده). پردازش‌ها از هم ایزوله هستن و ارتباط بینشون (Inter-Process Communication - IPC) سنگین و کند هست (مثل Socket، Pipe). سیستم عامل، Process رو مدیریت می‌کنه.

• ریس (Thread): به واحد اجرایی درون به پردازش است. هر Process حداقل به Thread داره (Thread اصلی یا main). تمام Thread‌های به Process، فضای حافظه مشترک دارن (همون Heap و داده‌ها)، اما هر کدوم Stack مخصوص به خودشون رو دارن. ارتباط بین Thread‌ها خیلی سبک‌تر و سریع‌تره (چون حافظه مشترکه). اما این اشتراک، مشکل هم‌روندی (Race Condition) رو ایجاد می‌کنه.

ویژگی	پردازه (Process)	ریسه (Thread)
حافظه	فضای مجزا و مستقل	فضای مشترک (Heap) + Stack مجزا
ارتباطات	کند و پیچیده (IPC)	سریع و ساده (حافظه مشترک)
ساخت	زمان بر و پرهزینه	سریع و کم هزینه
ایزوله بودن	ایزوله کامل	ایزوله نیستن (مشکل امنیتی/پایداری)
تعریف واقعی	اجرای یک برنامه (java -jar app.jar)	یک مسیر اجرا درون آن برنامه (هر ریکوئست در Tomcat)

7.2 مدیریت حافظه (Memory Management)

؟ سوال مصاحبه‌ای ۲: حافظه در یک Process چطور سازماندهی میشه؟ بخش‌های اصلی

حافظه (Memory Layout) رو نام ببر

جواب: وقتی یه برنامه (Process) اجرا میشه، سیستم عامل یه فضای حافظه مجازی بهش اختصاص میده که به بخش‌های زیر تقسیم میشه:

1. Text (Code Segment): شامل کدهای ماشین برنامه (دستورالعمل‌ها). معمولاً فقط خواندنی (Read-only).
2. Data Segment: شامل متغیرهای سراسری (Global) و ایستا (Static) که در طول عمر برنامه وجود دارن.
3. Stack (پشته): شامل متغیرهای محلی توابع، پارامترها و آدرس‌های بازگشت. هر Thread یه Stack مجزا داره. ساختار LIFO. سایز محدود (معمولاً چند مگابایت). پر شدنش باعث StackOverflowError میشه.
4. Heap (پشته): بزرگترین بخش حافظه که برای تخصیص حافظه پویا (Dynamic Memory Allocation) استفاده میشه. هر چیزی که در جاوا با new می‌سازی، توی Heap ساخته میشه. تمام Thread‌ها در یک Process به Heap مشترک دسترسی دارن. اینجاست که Garbage Collector جاوا کار می‌کنه.

7.3 همروندی و بن‌بست (Concurrency & Deadlock)

اینجا مکمل مباحث Concurrency در جاوا (فاز ۴) هست.

? سوال مصاحبه‌ای ۳: Deadlock (بن‌بست) چیه و ۴ شرط لازم برای رخ دادنش رو نام ببر.
(سوال کلاسیک سیستم عامل)

جواب: Deadlock وضعیتی که دو یا چند Process/Thread برای همیشه منتظر یکدیگر می‌مونن، چون هر کدوم منبعی رو در اختیار داره که دیگری بهش نیاز داره.

۴ شرط لازم (Coffman Conditions) که هر چهارتا باید همزمان برقرار باشن:

1. Mutual Exclusion (انحصار متقابل): حداقل یک منبع باید به صورت اشتراک‌ناپذیر باشه (در هر لحظه فقط یک Thread بتونه ازش استفاده کنه).

2. Hold and Wait (نگه‌داشتن و انتظار): یک Thread، حداقل یک منبع رو در اختیار داره و منتظر منابعی که در اختیار Thread های دیگه هست.

3. No Preemption (عدم قبضه کردن): سیستم عامل نمی‌تونه به زور منبع رو از یه Thread بگیره. Thread باید خودش داوطلبانه آزادش کنه.

4. Circular Wait (انتظار دایره‌ای): یه حلقه از Thread ها وجود داشته باشه که هرکدوم منتظر منبعی از Thread بعدی در حلقه باشه. (A منتظر B، B منتظر C، C منتظر A).

راه‌های مقابله (از دید برنامه‌نویس جاوا):

- همیشه قفل‌ها رو به یک ترتیب مشخص بگیریم.

- از tryLock() با زمان انتظار مشخص استفاده کن.

- از قفل‌های تو در تو تا حد امکان پرهیز کن.

7.4 زمان‌بندی (CPU Scheduling) CPU

? سوال مصاحبه‌ای ۴: تفاوت بین Context Switch در Process و Thread چیست؟

جواب: Context Switch عملیاتی که CPU از اجرای یک Process/Thread به Process/Thread دیگر سوئیچ می‌کند. این کار توسط Dispatcher در سیستم عامل انجام می‌شود.

- Context Switch پدیده: خیلی پرهزینه است. چون باید کل فضای حافظه (Page Tables)، وضعیت ثابت‌ها و ... عوض بشود. (Translation Lookaside Buffer) TLB در CPU هم باید Flush بشود.
- Context Switch ریسک: به مراتب سبک‌تر. چون Thread های یک Process، فضای حافظه مشترک دارند. فقط باید Stack Pointer و Program Counter و ثابت‌ها عوض بشوند. به همین خاطر Thread رو Lightweight Process هم می‌گویند.

7.5 مفاهیم سیستمی پراکنده ولی مفید

• Virtual Memory (حافظه مجازی): تکنیکی که سیستم عامل با کمک سخت‌افزار، حافظه اصلی (RAM) و حافظه جانبی (Disk) رو ترکیب می‌کند تا برنامه بتونه حافظه‌ای بیشتر از RAM فیزیکی موجود استفاده کند. باعث میشه هر Process تصور کنه که فضای حافظه بزرگ و پیوسته فقط برای خودش داره.

• Cache (حافظه پنهان): حافظه‌ای سریع و کوچک نزدیک CPU که برای کاهش تأخیر در دسترسی به داده‌های پرکاربرد از RAM استفاده میشه. در طراحی نرم‌افزار (مثل Redis, Ehcache) هم همین مفهوم به کار میره.

• Lock و Semaphore:

• Lock (قفل): به مکانیزم انحصاری. فقط یک Thread می‌تونه در یک زمان Lock رو بگیره.
(مثل ReentrantLock در جاوا).

• Semaphore (چراغ راهنما): به شمارنده است که اجازه می‌ده تعداد مشخصی Thread به طور همزمان به یک منبع دسترسی داشته باشند. (مثلاً اتصال به دیتابیس با حداکثر ۱۰ ارتباط همزمان).

این هم از فاز ۷. همونطور که می‌بینی، مفاهیم سیستم عامل مستقیماً روی درک ما از JVM، Multi-threading و حتی Performance Tuning تأثیر دارن.